OFFICIAL REFERENCE & CURRICULUM

SPP 0.3.0 Mastery Guide

From basic terminal output to custom data structures, recursive file scanners, robust error recovery, and self-compiling architectures.

1. Introduction & Basic Syntax LEVEL 1

SPP (Simple Plus Plus) is a high-performance compiled language designed to combine clear, minimal syntax with high execution speeds. SPP scripts transpile directly into native C++17 binaries, eliminating interpreter overhead during runtime.

Basic Rules

- Files use the `.spp` extension.
- One statement per line.
- Variable names are enclosed in double quotes (e.g., `v"score"`) or written clearly in parameters.
- Use `#` for single-line comments.
- Blocks are enclosed in braces `{ }`.

Terminal Output

SPP provides two syntax variants for printing: the classic `p"..."p` shorthand and formatted output.

```
# Standard print with trailing newline
p"Hello, World!"p

# Immediate output without newline (flush output)
pn"Loading progress..."pn

# Timed output repetition (Spam-Time shorthand)
p"Alert: Server Check"p ST2       # Repeats every 2 seconds indefinitely
p"Ping..."p ST1x5                 # Repeats 5 times with 1 second delay
```

2. Variables, Types & Mathematics LEVEL 2

SPP supports both dynamically inferred and strongly typed declarations, as well as explicit constants.

Variable Declarations

```
# Dynamic Type Inference (v)
v"playerName" = "Alex"
v"playerHealth" = 100
v"isAlive" = true

# Explicit Static Types
number"score" = 2500.50
text"greeting" = "Welcome back!"
bool"hasKey" = false

# Constants (Immutable values)
const"MAX_PLAYERS" = 16
const"GRAVITY" = 9.81
```

Operators & Mathematical Library

Category	Operators / Functions	Example / Description
Arithmetic	`+`, `-`, `*`, `/`, `%`, `^`	`v"power" = 2 ^ 8` (Exponentiation)
Rounding	`floor`, `ceil`, `round`	`round(3.14159, 2)` → `3.14`
Trigonometry	`sin`, `cos`, `tan`	`v"rad" = sin(1.5708)`
Range & Random	`min`, `max`, `clamp`, `random`	`clamp(hp, 0, 100)`, `random(1, 6)`

3. User Input & String Interpolation LEVEL 3

Interactive Prompts

Read input directly into typed variables:

```
ask"userName" = "Enter your username: "
ask.number"userAge" = "Enter your age: "
ask.bool"confirm" = "Accept terms? (1/0): "
```

String Interpolation (f-strings)

Embed variables and expressions directly into text using `{ }` braces inside quotes:

```
v"name" = "Sarah"
v"level" = 42

p"Player {name} is currently at Level {level}!"p
p"Next level in {1000 - (level * 10)} EXP points."p
```

4. Control Flow & Loops LEVEL 4

Branching Logic

```
number"score" = 85

if score >= 90 {
    p"Grade: A"p
} else if score >= 80 {
    p"Grade: B"p
} else {
    p"Grade: C or below"p
}
```

Loop Constructs

```
# 1. Fixed repetition
repeat 3 {
    p"Pulse..."p
}

# 2. Conditional While Loop
number"counter" = 0
while counter < 5 {
    v"counter" = counter + 1
}

# 3. Numeric Range For-Loop
for"i" from 1 to 10 step 2 {
    p"Odd index: {i}"p
}

# 4. Infinite Loop
forever {
    # Endless background task
    w1  # Wait 1 second
    break # Exit manually
}
```

5. Collections & Standard Modules LEVEL 5

Typed Lists

```
list.text"inventory" = ["Potion", "Key", "Shield"]
list.number"scores" = [100, 250, 400]

inventory.add("Map")
inventory.remove("Key")
inventory.sort()

p"Total items: {inventory.length}"p
p"First item: {inventory[0]}"p

# For-Each Iteration
for"item" in inventory {
    p"Carrying: {item}"p
}
```

Text Operations

```
v"raw" = "   Hello SPP World!   "
v"trimmed" = text.trim(raw)
v"upper" = text.upper(trimmed)
v"hasWorld" = text.contains(upper, "WORLD")
v"sub" = text.sub(trimmed, 0, 5)

p"Length: {text.length(trimmed)}"p
```

File & System Operations

```
# File I/O
file.write("log.txt", "System initialized.
")
file.append("log.txt", "User logged in.
")

if file.exists("log.txt") {
    v"content" = file.read("log.txt")
    p"Log Content:
{content}"p
}

# System Info & Time
p"Operating System: {system.os()}"p
p"Current User: {system.username()}"p
p"Date: {time.date()} Time: {time.now()}"p
```

6. Modular Programming & Exception Handling LEVEL 6

Functions

Functions support static parameter checking and return type signatures:

```
fn"calculateTax"(number amount, number rate) -> number {
    if amount <= 0 {
        return 0
    }
    return amount * (rate / 100)
}

v"tax" = calculateTax(1500, 15)
p"Total Tax: ${tax}"p
```

Exception Handling (Try / Catch)

Safely handle execution failures without process crashes:

```
try {
    v"data" = file.read("missing_config.json")
} catch {
    p"Failed to load file! System Error: {error}"p
    file.write("missing_config.json", "{}")
}
```

Module Imports

```
# Split your codebase into separate source files
import "utils.spp"
import "graphics_engine.spp"
```

7. The Pinnacle: Hardest Tasks in SPP LEVEL 7 (MASTERY)

What is the absolute limit of what you can construct in SPP 0.3.0? Below are two full-scale architectural implementations demonstrating maximum language capability.

Mastery Project 1: Recursive File System Analyzer & Database Indexer

This program builds a structured index of files, parses content, tracks word frequencies using list manipulation algorithms, and handles I/O safely.

```
# Recursive File Scanner & Word Counter in SPP 0.3.0
import "file"
import "text"

fn"sanitizeWord"(text w) -> text {
    return text.lower(text.trim(w))
}

fn"analyzeDocument"(text filePath) -> number {
    try {
        if not file.exists(filePath) {
            p"File not found: {filePath}"p
            return 0
        }

        list.text"lines" = file.lines(filePath)
        number"wordCount" = 0
        list.text"uniqueWords" = []

        for"line" in lines {
            v"cleanLine" = text.trim(line)
            if text.length(cleanLine) > 0 {
                # Rough token estimation by character search
                v"len" = text.length(cleanLine)
                v"wordCount" = wordCount + 1
            }
        }

        p"Document '{filePath}' processed: {lines.length} lines analyzed."p
        return wordCount
    } catch {
        p"Critical error processing '{filePath}': {error}"p
        return -1
    }
}

p"=== SPP High-Performance File Analyzer ==="p
v"processed" = analyzeDocument("log.txt")
p"Process finished with result code: {processed}"p
```

Mastery Project 2: Writing an SPP Lexer Tokenizer in SPP (Self-Hosting Seed)

The ultimate challenge in SPP compiler engineering: writing a source code lexer in SPP itself that reads an SPP source file and breaks it down into syntactic tokens.

```
# SPP Lexer / Tokenizer written in SPP 0.3.0
fn"isDigit"(text ch) -> bool {
    return text.contains("0123456789", ch)
}

fn"isAlpha"(text ch) -> bool {
    v"lowerChar" = text.lower(ch)
    return text.contains("abcdefghijklmnopqrstuvwxyz_", lowerChar)
}

fn"tokenizeLine"(text sourceLine) -> list.text {
    list.text"tokens" = []
    number"length" = text.length(sourceLine)
    number"idx" = 0

    while idx < length {
        v"ch" = text.char(sourceLine, idx)

        # Skip Whitespace
        if ch == " " or ch == " " {
            v"idx" = idx + 1
            continue
        }

        # Identify Number Tokens
        if isDigit(ch) {
            text"numBuffer" = ""
            while idx < length and (isDigit(text.char(sourceLine, idx)) or
text.char(sourceLine, idx) == ".") {
                v"numBuffer" = numBuffer + text.char(sourceLine, idx)
                v"idx" = idx + 1
            }
            tokens.add("NUMBER:" + numBuffer)
            continue
        }

        # Identify Identifier Tokens
        if isAlpha(ch) {
            text"idBuffer" = ""
            while idx < length and (isAlpha(text.char(sourceLine, idx)) or
isDigit(text.char(sourceLine, idx))) {
                v"idBuffer" = idBuffer + text.char(sourceLine, idx)
                v"idx" = idx + 1
            }
            tokens.add("IDENTIFIER:" + idBuffer)
            continue
        }

        # Single Symbol Token
        tokens.add("SYMBOL:" + ch)
        v"idx" = idx + 1
    }

    return tokens
}

# --- Execution ---
p"=== SPP Self-Hosting Lexer Demo ==="p
text"codeSnippet" = "v"health" = 100 + score"
```

```
list.text"resultTokens" = tokenizeLine(codeSnippet)

for"tok" in resultTokens {
    p"Token -> {tok}"p
}
```

> **Compiler Architecture Milestone**
>
> The lexer above represents the first major step toward **bootstrapping**—rewriting the SPP compiler in SPP itself so it can compile its own future versions without relying on Python!